

Medir sorpresa e incertidumbre: teoría de la información aplicada a secuencias

TC-CH07 – Técnicas Computacionales en Biología

Edición 2 · 2026-09-03

Pregunta del tema. Dos mensajes de cuatro letras ocupan lo mismo en disco y no dicen lo mismo. ¿Cuánta incertidumbre había antes de leerlos, y cómo se mide eso con un número?

Producto final. Un guion propio que calcula entropía por ventanas sobre una secuencia, con su contrato declarado y su comportamiento ante entradas que no son ADN, contrastado con un oráculo que no le da los valores de antemano.

Mapa del tema

1. Una pregunta cuantitativa antes de empezar	1
2. Del suceso a la sorpresa	1
3. De muchas sorpresas a una incertidumbre media	3
4. Ejemplos que pueden comprobarse	4
5. De una secuencia a una distribución empírica	5
6. Lo que la composición no puede mostrar	7
7. Cuando hay datos y también hay un modelo	9
8. Divergencia KL: cuantificar el desacuerdo	9
9. Información mutua: pasar de diversidad a dependencia	11
10. Una lectura unificada de las cinco cantidades	13
11. La escala total y el supuesto iid	13
12. Implementar sin ocultar el razonamiento	14
13. Límites de inferencia biológica	15
14. Mapa de decisión para resolver problemas	16
15. Síntesis y transición a modelos probabilísticos	16

1 Una pregunta cuantitativa antes de empezar

Imagine dos mensajes de cuatro letras: AAAA y ACGT. Tienen la misma longitud y ocupan el mismo número de caracteres en un archivo de texto, pero no producen la misma impresión. Si una fuente casi siempre emite A, el primer mensaje era previsible; si las cuatro bases eran igualmente probables, cada posición del segundo mensaje resulta menos anticipable. La longitud por sí sola no responde a la pregunta que abre este capítulo: *¿cuánta incertidumbre había antes de observar el mensaje?*

La teoría de la información permite formular esa pregunta sin decidir primero si el mensaje es útil, verdadero, funcional o biológicamente importante. Esta separación es una virtud y también un límite. Una medida matemática puede describir regularidades de los símbolos; no convierte automáticamente una región de baja entropía en un gen, un sitio regulador o una isla CpG. La interpretación biológica necesita evidencia adicional y un modelo adecuado al fenómeno.

Al terminar podrá distinguir tres objetos que suelen confundirse: una *distribución* declarada, una *secuencia* observada y una *medida* calculada a partir de ambas. También podrá explicar qué hipótesis hacen posible un cálculo, comprobar ejemplos manuales con Bash y detener una interpretación cuando la evidencia no permite ir más lejos.

2 Del suceso a la sorpresa

2.1 La probabilidad pertenece a un modelo

Antes de asignar información a una letra necesitamos declarar probabilidades. En una moneda ideal, por ejemplo, el modelo asigna $p(\text{cara}) = 1/2$. En una moneda sesgada podría asignar $p(\text{cara}) = 3/4$. En una secuencia, una

tabla como $p(A) = p(C) = p(G) = p(T) = 1/4$ no está escrita en la molécula: es una hipótesis, una estimación o una referencia elegida por quien analiza.

Esta observación evita una confusión frecuente. La probabilidad no es lo mismo que la frecuencia, aunque podamos estimarla mediante una frecuencia. Si aparecen 30 adeninas en 100 posiciones, la frecuencia observada es $30/100 = 0,30$. Usarla como estimación de $p(A)$ añade el supuesto de que esas observaciones representan la fuente que queremos estudiar. Cambiar el conjunto de datos, la región, la hebra o el mecanismo de muestreo puede cambiar la estimación.

2.2 Autoinformación

Buscamos una medida que aumente cuando disminuye la probabilidad. Además, si dos sucesos son independientes, queremos que la información conjunta sea la suma de las informaciones: observar dos caras independientes de probabilidad $1/2$ debería aportar el doble que observar una. El logaritmo satisface precisamente esta propiedad porque transforma productos en sumas.

$$I(x) = -\log_2 p(x) = \log_2 \left(\frac{1}{p(x)} \right). \quad (1)$$

La autoinformación de un resultado de probabilidad $p(x) > 0$ es $-\log_2 p(x)$ y se mide en bits cuando el logaritmo tiene base dos. El nombre *bit* no significa aquí que el archivo ocupe exactamente un bit: designa la unidad de la medida. Con logaritmos naturales la unidad sería el *nat*. No se deben mezclar resultados obtenidos con bases distintas sin convertirlos.

Conexión con la teoría computacional y árboles de decisión.

La relación entre $\log_2(1/p(x))$ y la unidad «bit» conecta formalmente con el límite inferior por árbol de decisión analizado en TC-CH06: si un suceso elemental tiene probabilidad $p(x) = 1/M$, hacen falta exactamente $\log_2 M$ preguntas binarias equilibradas (comparaciones dicotómicas) para aislar dicho suceso sin ambigüedad. Cada bit cuantifica la resolución de una decisión binaria ideal que divide por dos la incertidumbre del observador.

Tres valores construyen intuición. Si $p(x) = 1$, entonces $I(x) = 0$ bits: el resultado seguro no resuelve incertidumbre. Si $p(x) = 1/2$, aporta 1 bit. Si $p(x) = 1/8$, aporta 3 bits, pues harían falta tres decisiones binarias equilibradas para distinguir uno entre ocho resultados equiprobables.

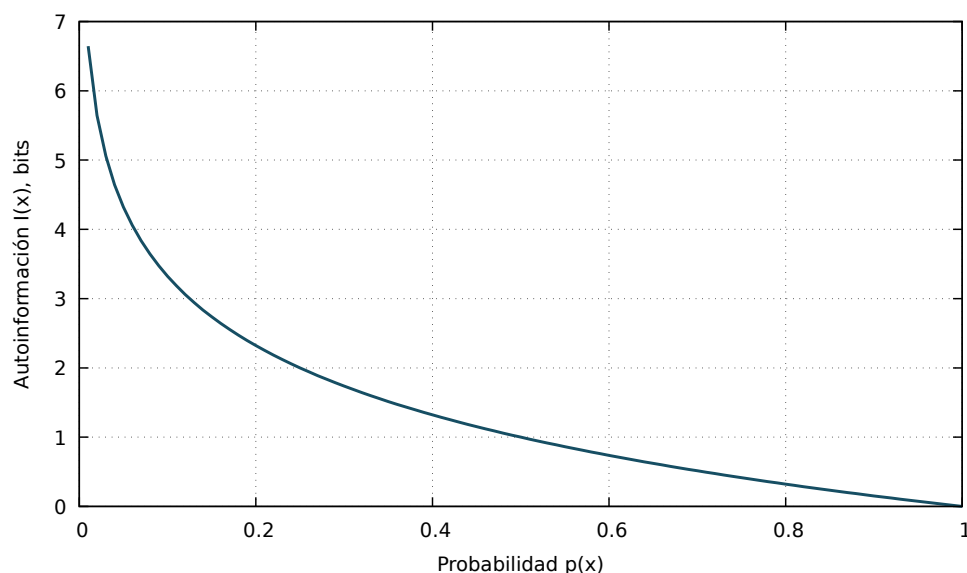


Figura 1: Autoinformación frente a probabilidad. En el dominio representado, $-\log_2 p(x)$ crece rápidamente al disminuir $p(x)$. Activo original TC07-F01, generado por `make_figures.sh selfinfo` desde TC07-F01-selfinfo.tsv.

El activo TC07-F01 se obtiene de cien probabilidades entre 0,01 y 1 mediante la función $-\log_2 p$ y muestra su crecimiento cuando p se aproxima a cero.

2.3 Por qué se suman las sorpresas independientes

Si x e y son independientes, su probabilidad conjunta es $p(x, y) = p(x)p(y)$. Sustituir ese producto en la definición y usar la regla del logaritmo da la aditividad.

$$I(x, y) = -\log_2[p(x)p(y)] = I(x) + I(y). \quad (2)$$

La autoinformación de dos resultados independientes es aditiva. La palabra *independientes* no es decorativa. Si conocer x cambia la probabilidad de y , el producto $p(x)p(y)$ deja de describir el suceso conjunto. Este punto regresará al estudiar información mutua.

Ejemplo resuelto: moneda sesgada.

En una moneda con $p(C) = 0,75$ y $p(X) = 0,25$, observar cara aporta $-\log_2(0,75) \approx 0,415$ bits; observar cruz aporta 2 bits. La cruz aporta más información cuando ocurre, pero ocurre menos. Para describir la incertidumbre media antes del lanzamiento tendremos que ponderar cada sorpresa por su probabilidad. Ese promedio conduce a la entropía.

Con probabilidades 0,75 y 0,25, las autoinformaciones de cara y cruz son aproximadamente 0,415037 y 2 bits, respectivamente.

3 De muchas sorpresas a una incertidumbre media

Sea X una variable aleatoria discreta que puede tomar valores en un alfabeto $\mathcal{X}$. Antes de observarla existe un abanico de resultados con sus probabilidades. Después de observar x , la sorpresa es $I(x)$. La media ponderada de todas esas sorpresas es la entropía.

La entropía de Shannon es la autoinformación esperada bajo una distribución declarada.

$$H(X) = \mathbb{E}[I(X)] = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x). \quad (3)$$

Para una variable discreta, la entropía de Shannon es $-\sum_x p(x) \log_2 p(x)$, con el convenio $0 \log 0 = 0$. El convenio para $p(x) = 0$ no afirma que $\log 0$ exista. Afirma que el límite de $-p \log p$ cuando p se acerca a cero es cero: un resultado que el modelo declara imposible no contribuye al promedio bajo ese mismo modelo.

3.1 Leer la fórmula sin convertirla en un ritual

Cada término tiene una función distinta. $-\log_2 p(x)$ mide la sorpresa de un resultado; $p(x)$ indica con qué frecuencia esperamos pagar esa sorpresa; la suma promedia sobre resultados incompatibles. Olvidar el factor $p(x)$ sumaría sorpresas sin ponderarlas. Olvidar el signo produciría números negativos. Usar conteos sin normalizarlos sustituiría probabilidades por cantidades que no suman uno.

La entropía es una propiedad de una distribución, no una etiqueta metafísica de una cadena. Cuando decimos «entropía de una secuencia», normalmente abreviamos «entropía de la distribución empírica obtenida al contar símbolos en esa secuencia». La abreviatura es cómoda, pero conviene recordar la operación.

3.2 Extremos y escala

$$0 \leq H(X) \leq \log_2 K, \quad (4)$$

Para una distribución sobre K resultados posibles, la entropía está entre 0 y $\log_2 K$ bits; el máximo se alcanza con la distribución uniforme. El mínimo corresponde a una fuente degenerada: un resultado tiene probabilidad uno. El máximo corresponde a K alternativas equiprobables. En un alfabeto de cuatro nucleótidos canónicos, la cota superior de orden cero es $\log_2 4 = 2$ bits por símbolo. La cota depende del alfabeto: si se admite también N , ya no es correcto seguir usando 2 como máximo teórico del alfabeto de cinco símbolos.

Que una distribución tenga más entropía no significa que sea «mejor». Una muestra contaminada puede aumentar la diversidad aparente; una región funcional puede ser conservada y mostrar menor variabilidad; una fuente

aleatoria puede tener entropía alta sin poseer una organización biológica interesante. La medida responde a una pregunta cuantitativa concreta, no a una escala universal de calidad.

```

ALGORITMO: Entropia_Shannon_Secuencia
ENTRADA:
- S: Cadena biologica de longitud L sobre alfabeto discreto Sigma (ej. {'A', 'C', 'G', 'T'})
SALIDA:
- H: Entropia de Shannon en bits (unidades base 2)
COMPLEJIDAD: Tiempo O(L + |Sigma|), Espacio auxiliar O(|Sigma|)

1. INICIALIZACION Y CONTEO:
  Si L == 0 entonces:
    Retornar ERROR_SECUENCIA_VACIA

  Para cada simbolo c en Sigma:
    Conteo[c] <- 0

  Para i desde 0 hasta L - 1:
    simbolo <- S[i]
    Conteo[simbolo] <- Conteo[simbolo] + 1

2. CALCULO DE LA ENTROPIA DE SHANNON:
  H <- 0.0
  Para cada c en Sigma:
    Si Conteo[c] > 0 entonces:
      p <- Conteo[c] / L          # Distribucion de probabilidad empirica
      H <- H - p * (log(p) / log(2)) # Base 2; si p=0 el termino se ignora por lim_{p->0} p*log(p) = 0

3. RETORNO:
  Retornar H  # 0.0 <= H <= log2(|Sigma|)

```

4 Ejemplos que pueden comprobarse

4.1 Cuatro símbolos equiprobables

Si $p(A) = p(C) = p(G) = p(T) = 1/4$, cada símbolo aporta $-\log_2(1/4) = 2$ bits. El promedio de cuatro contribuciones iguales sigue siendo 2.

$$H = -4 \left(\frac{1}{4} \right) \log_2 \left(\frac{1}{4} \right) = 2 \text{ bits/símbolo.} \quad (5)$$

Una distribución uniforme de cuatro símbolos tiene entropía de 2 bits por símbolo. La unidad «por símbolo» recuerda que hemos calculado un promedio. Bajo un modelo independiente e idénticamente distribuido, 12 posiciones tendrían una entropía conjunta de $12 \times 2 = 24$ bits. Esa multiplicación ya incorpora un modelo sobre la relación entre posiciones; no se deduce solo de ver una cadena de longitud 12.

Bajo un modelo iid uniforme de cuatro símbolos, doce posiciones tienen entropía conjunta de 24 bits.

4.2 Una moneda sesgada

La moneda del apartado anterior tiene una cara frecuente y una cruz rara. Su entropía pondera ambas sorpresas:

$$H = -0,75 \log_2(0,75) - 0,25 \log_2(0,25) \approx 0,811278 \text{ bits/lanzamiento.} \quad (6)$$

Una variable binaria con probabilidades 0,75 y 0,25 tiene entropía aproximada de 0,811278 bits. El resultado es menor que el máximo binario de 1 bit. No porque la cruz deje de ser sorprendente, sino porque la fuente suele emitir el resultado previsible.

4.3 AAAA frente a ACGT

Para convertir una cadena en distribución empírica contamos sus símbolos y dividimos por la longitud. En AAAA, $\hat{p}(A) = 1$ y las demás frecuencias son cero: la entropía de orden cero es 0. En ACGT, las cuatro frecuencias

son 1/4: la entropía es 2 bits por símbolo.

$$\begin{array}{|c|c|c|c|} \hline A & A & A & A \\ \hline \end{array} \quad \hat{H}_0 = 0 \quad \begin{array}{|c|c|c|c|} \hline A & C & G & T \\ \hline \end{array} \quad \hat{H}_0 = 2$$

Figura 2: Dos cadenas de igual longitud producen distribuciones empíricas distintas. Composición del capítulo TC07-G01; los valores se reproducen con el caso sequences del oráculo Bash.

Las distribuciones empíricas de AAAA y ACGT tienen entropías de orden cero de 0 y 2 bits por símbolo, respectivamente.

Este ejemplo separa longitud y composición, pero todavía no dice nada sobre el orden. Para ver ese límite necesitamos un par construido con los mismos conteos.

5 De una secuencia a una distribución empírica

5.1 Contar, normalizar y agregar

Sea una secuencia de longitud n y n_x el número de apariciones del símbolo x . La estimación más directa sustituye la probabilidad desconocida por su frecuencia relativa.

$$\hat{p}(x) = \frac{n_x}{n}, \quad \hat{H}_0 = - \sum_{x:n_x>0} \hat{p}(x) \log_2 \hat{p}(x). \quad (7)$$

El estimador empírico de orden cero se obtiene contando cada símbolo, normalizando por la longitud y aplicando la fórmula de Shannon a las frecuencias relativas no nulas. El subíndice cero indica que el estado utilizado para predecir el símbolo siguiente no incluye símbolos anteriores. No significa «entropía aproximada de mala calidad»: identifica el orden del modelo.

En Bash, la tubería natural refleja las tres operaciones. Bash organiza la entrada, la salida y los errores; awk recorre caracteres, mantiene un arreglo de conteos y evalúa logaritmos. No hace falta introducir otro lenguaje curricular.

5.2 Puente de Shell a awk: una transformación cada vez

Este capítulo *recupera* destrezas cuyos propietarios son TC-CH01, *Entorno Linux, Shell y reproducibilidad* (ejecutar un archivo y registrar su entorno); TC-CH02, *Bash, automatización y procesamiento de texto* (argumentos, tuberías, expansión y awk); y TC-CH03, *Algoritmos, complejidad y estrategias de resolución* (entrada, estado, traza y oráculo). Si no puede anticipar qué texto recibe cada lado de |, conviene recuperar TC-CH02 antes del script completo.

Este capítulo recupera destrezas propiedad de TC-CH01 (ejecutar un archivo y registrar su entorno), TC-CH02 (argumentos, tuberías, expansión y awk) y TC-CH03 (entrada, estado, traza y oráculo), sin volver a enseñarlas.

El siguiente puente solo cuenta. Bash guarda AACG en sequence; printf la convierte en una línea; la tubería entrega esa línea a awk; el primer bloque actualiza count; y END imprime una tabla en el orden fijo A, C, G, T. Fijar LC_ALL=C estabiliza el punto decimal y la comparación de caracteres.

```
#!/usr/bin/env bash
set -euo pipefail
sequence=${1:-AACG}
printf '%s\n' "$sequence" |
LC_ALL=C awk '
{ for (i=1; i<=length($0); i++) count[substr($0,i,1)]++ }
END {
    print "symbol\tcount"
    for (i=1; i<=4; i++) { s=substr("ACGT",i,1); printf "%s\t%d\n",s,count[s]+0 }
}'
```

El programa bridge_counts.sh AACG imprime los conteos $A = 2$, $C = 1$, $G = 1$ y $T = 0$ en ese orden.

ESENCIAL El algoritmo completo añade contrato de entrada, normalización y entropía. Acepta una o más cadenas como argumentos separados; exige mayúsculas y el alfabeto ACGT; una entrada vacía o inválida termina con

estado no cero. Recorre siempre ACGT, por lo que la salida no depende del orden interno de los arreglos asociativos de awk.

```
#!/usr/bin/env bash
set -euo pipefail

if (( $# == 0 )); then
    printf 'usage: %s SEQUENCE [SEQUENCE ...]\n' "$0" >&2
    exit 64
fi

for sequence in "$@"; do
    if [[ -z "$sequence" ]]; then
        printf 'ERROR: empty sequence\n' >&2
        exit 65
    fi
    if [[ ! "$sequence" =~ ^[ACGT]+$ ]]; then
        printf 'ERROR: expected uppercase A C G or T: %s\n' "$sequence" >&2
        exit 66
    fi
    LC_ALL=C awk -v sequence="$sequence" '
        function log2(x) { return log(x)/log(2) }
        BEGIN {
            for (i=1; i<=length(sequence); i++) count[substr(sequence,i,1)]++
            for (i=1; i<=4; i++) {
                symbol=substr("ACGT",i,1)
                if (count[symbol]>0) {
                    p=count[symbol]/length(sequence)
                    h-=p*log2(p)
                }
            }
            printf "%s\t%.6f\n", sequence, h
        }'
done
```

Una lectura cuidadosa del programa distingue estado y cálculo. count guarda los conteos; n guarda la longitud; el bucle final normaliza antes de acumular. Solo se recorren símbolos observados, así que nunca se evalúa $\log(0)$. El formato con seis decimales es una decisión de presentación, no una precisión experimental garantizada.

Traza manual.

Para AACG, los conteos son $n_A = 2$, $n_C = 1$, $n_G = 1$. Las frecuencias son $1/2$, $1/4$ y $1/4$. Las contribuciones son 0,5, 0,5 y 0,5 bits, de modo que $\hat{H}_0 = 1,5$ bits por símbolo. La suma de contribuciones, no la media aritmética de tres valores, ya contiene la ponderación por probabilidad.

La distribución empírica de AACG es $(1/2, 1/4, 1/4, 0)$ sobre A, C, G, T y su entropía de orden cero es 1,5 bits por símbolo.

5.3 Precondiciones que el cálculo debe declarar

Un script fiable no «limpia» silenciosamente cualquier entrada. Antes de contar hay que decidir al menos:

- si minúsculas y mayúsculas representan el mismo símbolo;
- si se eliminarán espacios y saltos de línea de un FASTA;
- si las cabeceras que empiezan por > se excluyen;
- si N pertenece al alfabeto, se ignora o provoca error;
- qué hacer con una secuencia vacía;
- si se analiza una sola secuencia o se agrupan varias.

Cada elección cambia el objeto estadístico. Ignorar N cambia el denominador; tratarla como quinto símbolo cambia el alfabeto; repartirla entre las cuatro bases inventa fracciones que no fueron observadas. Ninguna

política es universalmente correcta, pero debe quedar escrita y ser coherente con la pregunta.

El contrato publicado de `entropy0.sh` preserva el orden de varios argumentos válidos y rechaza ausencia de argumentos, cadenas vacías y símbolos fuera de ACGT con un estado de salida distinto de cero.

REFERENCIA

5.4 Frecuencia observada no equivale a verdad poblacional

La entropía obtenida al insertar frecuencias observadas es un estimador (*plug-in*) y puede presentar sesgo apreciable cuando el número de observaciones es pequeño respecto del número de categorías.

Supongamos una fuente generadora ideal con $P(A) = P(C) = P(G) = P(T) = 1/4$, cuya entropía poblacional es exactamente 2 bits por símbolo. Las muestras cortas "AAAA" y "ACGT" son ambas realizaciones perfectamente posibles bajo esta misma fuente uniforme:

- Para la muestra "AAAA", la probabilidad de emisión bajo la fuente uniforme es $(1/4)^4 = 1/256$: un suceso poco frecuente, pero válido. Sin embargo, su estimación empírica directa (*plug-in*) produce $\hat{H}_0 = 0$ bits/símbolo.
- Para la muestra "ACGT", la estimación *plug-in* produce $\hat{H}_0 = 2$ bits/símbolo.

Una muestra finita con frecuencias observadas cero para algunas bases no redefine la fuente estocástica subyacente. En bioinformática rigurosa es imperativo reportar el tamaño de muestra n , el alfabeto considerado y el estimador utilizado antes de inferir propiedades poblacionales.

6 Lo que la composición no puede mostrar

Compare AAAACCCC con ACACACAC. Ambas cadenas contienen cuatro A y cuatro C. Su distribución empírica de símbolos es $(1/2, 1/2)$ y por tanto su entropía de orden cero es 1 bit por símbolo. Sin embargo, la primera agrupa símbolos y la segunda alterna con regularidad.

$$\text{AAAACCCC} \rightarrow \{A : 4, C : 4\} \leftarrow \text{ACACACAC}$$

Mismo histograma, distinto orden secuencial

Figura 3: La entropía de orden cero factoriza el orden: conserva conteos y descarta la posición de cada aparición. Figura original.

Dos secuencias con la misma composición tienen la misma entropía de orden cero aunque difieran en el orden de sus símbolos. No se trata de un fallo de implementación: es una consecuencia exacta de la entrada elegida por el modelo. La fórmula recibe $p(A)$ y $p(C)$, no recibe la lista de posiciones. Ningún algoritmo puede recuperar de esos dos números qué cadena los produjo.

La consecuencia didáctica es profunda. «Entropía alta» no equivale a «secuencia sin patrones», porque un patrón puede conservar una composición equilibrada. ACACACAC es perfectamente predecible después de conocer un símbolo, pero su entropía de orden cero alcanza el máximo del alfabeto binario. Para representar esa dependencia se necesitan pares, contextos o estados: precisamente la puerta de entrada a modelos de Markov y HMM en el capítulo siguiente.

La cadena alternante ACACACAC tiene un bit de entropía de orden cero por símbolo, pero entropía condicional de primer orden cero porque cada A va seguida de C y cada C va seguida de A.

6.1 Entropía por ventanas: localizar sin diagnosticar

Una sola distribución para una secuencia larga mezcla regiones potencialmente distintas. Una ventana de ancho w calcula la distribución empírica en posiciones $i, \dots, i + w - 1$ y desplaza el inicio. Así obtenemos un perfil local. Con la secuencia sintética AAAACGTACGT y $w = 4$, la primera ventana AAAA tiene entropía 0; al entrar otros símbolos, la entropía aumenta; las ventanas ACGT alcanzan 2.

En el ejemplo sintético AAAACGTACGT con ventanas de cuatro símbolos, el perfil de entropía de orden cero pasa de 0 en AAAA a 2 en las ventanas ACGT; la serie completa es 0, 0, 0,811278, 1,5, 2, 2, 2, 2 y 2.

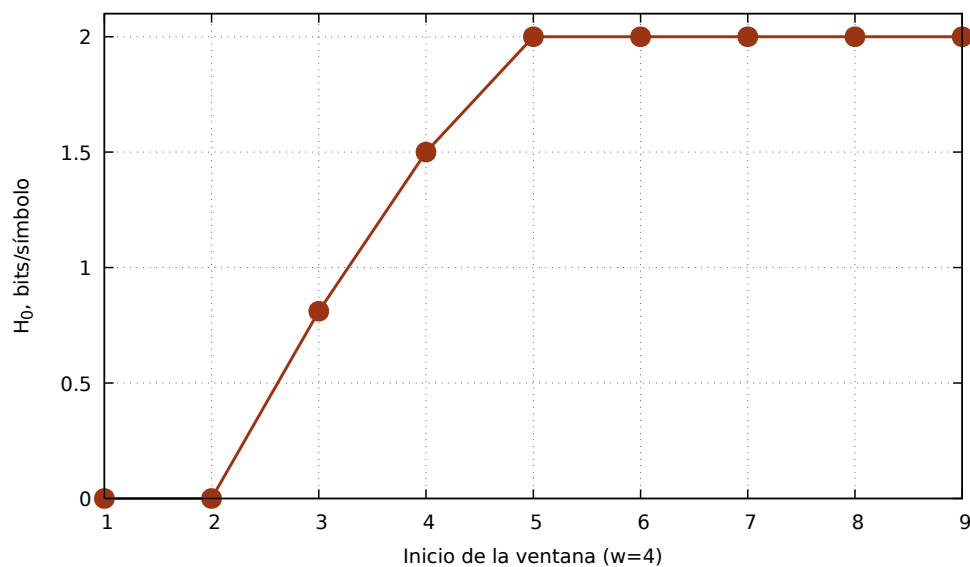


Figura 4: Perfil reproducible de AAAAACGTACGT con ancho cuatro. Activo original TC07-F03, generado por `make_figures.sh windows` desde `TC07-F03-windows.tsv`. Los puntos son ventanas discretas; la línea solo guía su lectura.

El ancho controla un compromiso. Ventanas pequeñas localizan cambios con mayor precisión, pero sus frecuencias son inestables y solo pueden tomar pocos valores. Ventanas grandes estabilizan conteos, pero suavizan fronteras. Además, ventanas solapadas no son observaciones independientes: comparten casi todos sus símbolos. Por ello, un perfil sirve para formular candidatos y comparar regiones bajo una regla declarada; no constituye por sí solo una prueba de función.

Al pasar de ancho cuatro a ancho ocho, el paso posible de las frecuencias baja de $1/4$ a $1/8$ y el solapamiento entre ventanas consecutivas sube de tres a siete símbolos. Bajo muestreo iid con $p = 1/2$, la varianza de la frecuencia estimada, $p(1 - p)/w$, baja de 0,0625 a 0,03125: se gana estabilidad de conteo a costa de localizar las fronteras en una región más ancha.

Un perfil de entropía por ventanas puede priorizar regiones candidatas bajo una regla declarada, pero no demuestra por sí solo función biológica ni satisface los campos de una anotación.

En una ventana de ancho w , las frecuencias cambian en pasos de $1/w$ y dos ventanas consecutivas desplazadas una posición comparten $w - 1$ símbolos; por ello su perfil no contiene observaciones independientes.

En particular, una ventana de baja entropía no identifica automáticamente un gen, una isla CpG o una región reguladora. Para una isla CpG se emplean propiedades como contenido GC y frecuencia observada/esperada del dinucleótido CpG, junto con una definición operativa y contexto genómico. La entropía puede añadir una descripción, pero no sustituye esas variables ni valida una anotación.

```

ALGORITMO: Perfil_Entropia_Ventana_Deslizante
ENTRADA:
- S: Cadena genómica de longitud L
- W: Ancho de ventana en nucleótidos ( $1 \leq W \leq L$ )
- Paso: Desplazamiento entre ventanas consecutivas (típicamente Paso = 1)
SALIDA:
- Perfil: Lista de tuplas (posicion_inicio, entropia_H)
COMPLEJIDAD: Tiempo  $O((L - W) / \text{Paso} + 1) * W$ 

1. INICIALIZACION:
  Perfil <- Lista_Vacia()
  pos <- 0

2. RECORRIDO DE VENTANAS DESLIZANTES:
  Mientras pos + W <= L:
    subcadena_ventana <- S[pos .. pos + W - 1]
    H_ventana <- Entropia_Shannon_Secuencia(subcadena_ventana)

```



```

Perfil.APPEND((pos, H_ventana))
pos <- pos + Paso

```

3. RETORNO:
Retornar Perfil

La regla operativa histórica de una isla CpG combina una longitud superior a 200 pares de bases, contenido GC superior al 50% y cociente CpG observado/esperado superior a 0,6; por ello una entropía baja no satisface esa definición [5].

7 Cuando hay datos y también hay un modelo

Hasta ahora una sola distribución ha ocupado el centro. En la práctica suelen aparecer dos: P , que representa la fuente de datos o la distribución que queremos describir, y Q , un modelo usado para anticiparlos. Preguntar «¿cuánta incertidumbre tiene P ?» no es lo mismo que preguntar «¿qué sorpresa media sufro si codifico datos de P como si procedieran de Q ?» La segunda pregunta define la entropía cruzada.

$$H(P, Q) = - \sum_x p(x) \log_2 q(x). \quad (8)$$

La entropía cruzada $H(P, Q)$ es la sorpresa media bajo datos distribuidos como P cuando las probabilidades usadas para evaluarlos son las de Q . La asimetría de papeles se ve en la fórmula: $p(x)$ pondera qué observaciones ocurren; $q(x)$ determina la penalización asignada. Intercambiar P y Q cambia ambas funciones y puede cambiar el resultado.

Ejemplo resuelto.

Supongamos dos resultados. Los datos siguen $P = (0,5, 0,5)$, mientras el modelo afirma $Q = (0,75, 0,25)$. La entropía de los datos es 1 bit. Evaluarlos con Q produce

$$H(P, Q) = -\frac{1}{2} \log_2(0,75) - \frac{1}{2} \log_2(0,25) \approx 1,207519 \text{ bits}. \quad (9)$$

La diferencia, unos 0,207519 bits por observación, cuantifica el exceso medio asociado a usar ese modelo en vez de la distribución P .

Para $P = (0,5, 0,5)$ y $Q = (0,75, 0,25)$ se obtiene $H(P) = 1$, $H(P, Q) \approx 1,207519$ y un exceso $D_{\text{KL}}(P\|Q) \approx 0,207519$ bits.

Contrato numérico de distribuciones de probabilidad.

Las funciones de cálculo de entropía cruzada y divergencia aceptan valores decimales no negativos representados con punto (por ejemplo 0, 0.25 o 1.0), pertenecientes al intervalo cerrado $[0, 1]$ y cuya suma total equivalga a 1 dentro de una tolerancia numérica explícita ($|1 - \sum p_i| \leq 10^{-6}$). No se aceptan comas decimales ni signos negativos en este validador.

8 Divergencia KL: cuantificar el desacuerdo

La divergencia de Kullback–Leibler compara las mismas dos distribuciones y conserva la dirección «datos P frente a modelo Q ».

$$D_{\text{KL}}(P\|Q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)} = H(P, Q) - H(P). \quad (10)$$

Cuando Q asigna probabilidad positiva a todo resultado posible bajo P , la divergencia KL satisface $D_{\text{KL}}(P\|Q) = H(P, Q) - H(P)$. La identidad permite leer KL como una penalización media por desajuste. No dice que una muestra finita ocupe literalmente ese número de bits en un archivo. Conecta cantidades ideales bajo distribuciones y, para interpretaciones de codificación, requiere el marco y los límites apropiados.

8.1 No negatividad no significa distancia

La divergencia KL es no negativa y vale cero si y solo si las dos distribuciones coinciden en el soporte relevante. Que sea no negativa la hace útil como medida de desajuste, pero no basta para ser una distancia métrica. Una distancia debe ser simétrica y obedecer además la desigualdad triangular. KL falla ya en la primera condición.

Retomemos $P = (0,5,0,5)$ y $Q = (0,75,0,25)$. El cálculo anterior da $D_{KL}(P||Q) \approx 0,207519$ bits. En la dirección inversa, $D_{KL}(Q||P) \approx 0,188722$ bits.

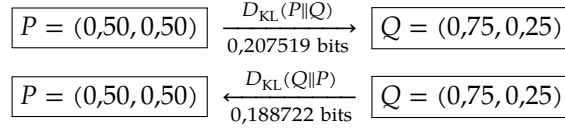


Figura 5: La dirección forma parte de la pregunta. Figura original; valores reproducidos por `verification/chapter_results.sh crosskl`. Activo TC07-G02.

La divergencia de Kullback–Leibler no es, en general, simétrica. Por eso conviene evitar la expresión «distancia KL» salvo que se esté explicando con precisión un uso no métrico. Promediar ambas direcciones crea una divergencia simétrica, pero la simetría por sí sola tampoco demuestra la desigualdad triangular. El nombre correcto en este capítulo es *divergencia*.

8.2 El caso límite de un cero en el modelo

Si $p(x) > 0$ pero $q(x) = 0$, el modelo declara imposible un resultado que la fuente puede producir. La sorpresa $-\log_2 q(x)$ diverge y también lo hace KL.

$$p(x) > 0 \text{ y } q(x) = 0 \implies D_{KL}(P||Q) = +\infty. \quad (11)$$

Si P asigna masa positiva a un resultado al que Q asigna cero, la divergencia $D_{KL}(P||Q)$ es infinita. No debe «arreglarse» el problema omitiendo el término: hacerlo cambia la definición y oculta un conflicto esencial entre datos y modelo. Una alternativa práctica es suavizar Q con pseudoconteos, pero el parámetro de suavizado pasa a formar parte del análisis y debe comunicarse.

REFERENCIA

8.3 Pseudoconteos: útiles, no invisibles

Supongamos conteos $(A, C, G, T) = (3, 1, 0, 0)$. La estimación directa asigna cero a G y T. Si añadimos un pseudoconteo de Laplace $\alpha = 1$ a las cuatro categorías, el denominador pasa de 4 a 8 y obtenemos $Q = (4/8, 2/8, 1/8, 1/8)$. Su entropía es 1,75 bits por símbolo.

$$\tilde{p}(x) = \frac{n_x + \alpha}{n + K\alpha}, \quad (3, 1, 0, 0), \alpha = 1 \implies \tilde{H} = 1,75. \quad (12)$$

Añadir $\alpha = 1$ a los conteos $(3, 1, 0, 0)$ produce probabilidades $(0,5, 0,25, 0,125, 0,125)$ y entropía de 1,75 bits. El pseudoconteo impide ceros, pero también desplaza todas las probabilidades hacia la uniformidad. Con pocos datos, el desplazamiento puede ser grande. Por eso hay que comparar decisiones razonables de α y no presentar el valor suavizado como si fuera un conteo observado.

En el suavizado $\tilde{p}(x) = (n_x + \alpha)/(n + K\alpha)$ con $\alpha > 0$, aumentar α acerca cada probabilidad a $1/K$; el efecto relativo es mayor cuando el total observado n es pequeño.

Para los conteos $(3, 1, 0, 0)$, al usar $\alpha = 0,5, 1$ y 2 , la probabilidad suavizada de A baja de 0,583333 a 0,5 y 0,416667, mientras la entropía sube de 1,551098 a 1,75 y 1,887919 bits; una conclusión dependiente de ese cambio debe declararse sensible al pseudoconteo.

8.4 Un procedimiento de comparación reproducible

Para comparar una distribución empírica P y un modelo Q conviene seguir un orden fijo:

1. declarar el mismo alfabeto y el orden de sus categorías;
2. comprobar que cada distribución es no negativa y suma uno dentro de una tolerancia explícita;
3. inspeccionar si existe algún $p(x) > 0$ con $q(x) = 0$;
4. calcular primero $H(P)$ y $H(P, Q)$;
5. calcular KL de forma directa y comprobar la identidad $H(P, Q) - H(P)$;
6. informar siempre la dirección y la base del logaritmo.

Este doble cálculo es un oráculo interno. Si la suma directa de KL no coincide con la diferencia de entropías más allá del redondeo esperado, hay un error en el orden de categorías, en la normalización, en la base del logaritmo o en el manejo de ceros.

8.5 Programa guiado para P , Q y KL

ESTUDIO El archivo siguiente recibe cuatro números: dos probabilidades para P y dos para Q . `awk` valida números no negativos y normalización, detiene el caso de soporte incompatible y conserva en variables separadas $H(P)$, $H(P, Q)$ y KL. La fila `CHECK` calcula la misma divergencia por la identidad de la ecuación 10; la coincidencia de ambas filas es el oráculo. Aquí el andamiaje aún nombra cada cantidad antes de pedir autonomía.

```
#!/usr/bin/env bash
set -euo pipefail

if (( $# != 4 )); then
    printf 'usage: %s P0 P1 Q0 Q1\n' "$0" >&2
    exit 64
fi

LC_ALL=C awk -v p0="$1" -v p1="$2" -v q0="$3" -v q1="$4" '
function log2(x) { return log(x)/log(2) }
function valid_number(x) { return x ~ /^[0-9]+(\.[0-9]*)?/[0-9]+$/ }
BEGIN {
    if (!valid_number(p0)||!valid_number(p1)||!valid_number(q0)||!valid_number(q1)) exit 65
    if (p0<0||p1<0||q0<0||q1<0) exit 66
    if ((p0+p1-1)^2>1e-12||((q0+q1-1)^2>1e-12)) exit 67
    if ((p0>0&&q0==0)||((p1>0&&q1==0))) exit 68
    p[0]=p0; p[1]=p1; q[0]=q0; q[1]=q1
    for (i=0; i<2; i++) if (p[i]>0) {
        hp-=p[i]*log2(p[i]); hpq-=p[i]*log2(q[i]); d+=p[i]*log2(p[i]/q[i])
    }
    printf "H_P\t%.6f\nH_PQ\t%.6f\nKL_P_Q\t%.6f\nCHECK\t%.6f\n", hp, hpq, d, hpq-hp
}'
```

El programa `compare_pq.sh 0.5 0.5 0.75 0.25` devuelve 1,000000, 1,207519, 0,207519 y 0,207519 para $H(P)$, $H(P, Q)$, KL y su comprobación, respectivamente.

9 Información mutua: pasar de diversidad a dependencia

La entropía marginal de una variable no describe cómo se relaciona con otra. Podemos tener dos posiciones individualmente equilibradas y, sin embargo, perfectamente ligadas. La información mutua compara la distribución conjunta $p(x, y)$ con la que existiría si las variables fueran independientes, $p(x)p(y)$.

$$I(X; Y) = \sum_x \sum_y p(x, y) \log_2 \frac{p(x, y)}{p(x)p(y)}. \quad (13)$$

La información mutua es la divergencia KL entre la distribución conjunta P_{XY} y el producto de marginales $P_X P_Y$. Al sustituir $Q(x, y) = p(x)p(y)$ en la fórmula de KL aparece exactamente el sumando de la ecuación 13; la demostración enlazada en el registro nombra esta identidad. Responde a una pregunta precisa: ¿cuánto cambia la descripción de una variable al conocer la otra?

9.1 Tabla conjunta mínima: dependencia perfecta

Considere dos variables binarias con solo dos pares observables: $(0,0)$ y $(1,1)$, cada uno con probabilidad $1/2$. Ambas marginales son uniformes, pero conocer X determina Y .

	$Y = 0$	$Y = 1$	$p_X(x)$
$X = 0$	$1/2$	0	$1/2$
$X = 1$	0	$1/2$	$1/2$
$p_Y(y)$	$1/2$	$1/2$	1

Tabla 1: Composición del capítulo TC07-G03: tabla conjunta de dependencia perfecta.

En cada celda no nula, el cociente es $(1/2)/[(1/2)(1/2)] = 2$ y su logaritmo es 1. Dos contribuciones de $1/2$ suman 1 bit. Si, en cambio, las cuatro celdas valen $1/4$, cada cociente es uno y la información mutua vale cero.

Una tabla binaria con $p(0,0) = p(1,1) = 1/2$ tiene 1 bit de información mutua; la tabla uniforme con cuatro celdas de $1/4$ tiene 0 bits.

La implementación retira parte del andamiaje anterior: ya no recibe marginales, sino cuatro celdas y debe construir p_X y p_Y . El doble bucle calcula a la vez MI y KL frente al producto; sus dos filas deben coincidir.

El programa de información mutua retira el andamiaje de recibir marginales ya calculadas: a partir de cuatro celdas debe construir p_X y p_Y y calcular en el mismo recorrido MI y KL frente al producto de marginales, con resultados que deben coincidir entre ambos.

```
#!/usr/bin/env bash
set -euo pipefail

if (( $# != 4 )); then
    printf 'usage: %s P00 P01 P10 P11\n' "$0" >&2
    exit 64
fi

LC_ALL=C awk -v p00="$1" -v p01="$2" -v p10="$3" -v p11="$4" '
function log2(x) { return log(x)/log(2) }
function valid_number(x) { return x ~ /^[0-9]+([.][0-9]*)?|([.][0-9]+)/ }
BEGIN {
    if (!valid_number(p00)||!valid_number(p01)||!valid_number(p10)||!valid_number(p11)) exit 65
    p[0,0]=p00; p[0,1]=p01; p[1,0]=p10; p[1,1]=p11
    total=0
    for (x=0;x<2;x++) for (y=0;y<2;y++) { if (p[x,y]<0) exit 66; total+=p[x,y] }
    if ((total-1)^2>1e-12) exit 67
    for (x=0;x<2;x++) for (y=0;y<2;y++) { px[x]+=p[x,y]; py[y]+=p[x,y] }
    for (x=0;x<2;x++) for (y=0;y<2;y++) if (p[x,y]>0) {
        product=px[x]*py[y]
        contribution=p[x,y]*log2(p[x,y]/product)
        mi+=contribution; kl+=contribution
    }
    printf "MI\t%.6f\nKL_JOINT_PRODUCT\t%.6f\n",mi,kl
}'
```

El programa `mutual_info_2x2.sh 0.5 0 0 0.5` devuelve 1 bit tanto como información mutua como KL entre la conjunta y el producto de marginales.

Para variables discretas, la información mutua es no negativa y es cero si y solo si las variables son independientes. «Cero» es una afirmación sobre la distribución ideal. Una estimación finita puede ser positiva incluso si el mecanismo es independiente, especialmente cuando hay pocas observaciones respecto de las celdas posibles. Por eso un valor estimado requiere incertidumbre, controles y una comparación nula apropiada antes de convertirse en evidencia de asociación.

Si X e Y son binarias, uniformes e independientes, dos observaciones pueden producir por azar la tabla empírica diagonal $(0,0)$, $(1,1)$ —en cualquier orden— con probabilidad $1/8$; su estimador *plug-in* de información mutua

vale 1 bit pese al mecanismo independiente.

Una información mutua empírica positiva necesita compararse con la variabilidad de muestreo o con un control nulo apropiado antes de interpretarse como evidencia de asociación.

9.2 Qué constituye una muestra para posiciones de secuencia

Para calcular información mutua entre dos posiciones no basta con elegir dos letras de una única cadena. Se necesita una colección de observaciones emparejadas: por ejemplo, columnas homólogas de un alineamiento de múltiples secuencias, o pares de posiciones obtenidos bajo un diseño de muestreo declarado. Cada secuencia aporta entonces un par (x_i, y_i) a la tabla conjunta.

Calcular información mutua entre dos posiciones exige una colección de observaciones emparejadas —por ejemplo columnas homólogas de un alineamiento múltiple o pares de posiciones bajo un diseño de muestreo declarado— y no dos letras tomadas de una única cadena.

Incluso con un alineamiento, la interpretación requiere prudencia. Secuencias muy relacionadas por ascendencia no forman necesariamente observaciones independientes; la calidad del alineamiento condiciona qué posiciones se comparan; huecos y símbolos ambiguos necesitan una política; múltiples pares de columnas crean un problema de multiplicidad. La información mutua puede señalar dependencia estadística, pero no demuestra contacto físico, coevolución directa ni causalidad.

La información mutua es simétrica, $I(X; Y) = I(Y; X)$; un único valor simétrico no identifica una dirección causal.

En alineamientos de proteínas, la información mutua cruda puede contener ruido de fondo y componentes filogenéticos, por lo que una asociación entre columnas no constituye por sí sola evidencia de contacto directo [3].

10 Una lectura unificada de las cinco cantidades

Las fórmulas se recuerdan mejor si se asocian a preguntas, no a nombres aislados.

Cantidad	Pregunta	Objeto que debe declararse
Autoinformación	¿Cuánto sorprende este resultado?	Una probabilidad $p(x)$
Entropía	¿Cuál es la sorpresa media de la fuente?	Una distribución P
Entropía cruzada	¿Cómo evalúa Q datos de P ?	Dos distribuciones y sus papeles
KL	¿Qué exceso medio introduce el desajuste?	Dirección $P\ Q$ y soporte
Información mutua	¿Cuánto se aparta la conjunta de la independencia?	Conjunta y marginales

Autoinformación, entropía, entropía cruzada, KL e información mutua responden a preguntas distintas y no son intercambiables.

Una estrategia útil ante cualquier problema es escribir primero la frase que se quiere cuantificar. Si la frase contiene «un resultado», piense en autoinformación. Si contiene «promedio de una fuente», piense en entropía. Si contiene «datos bajo un modelo», distinga entropía cruzada y KL. Si contiene «dependencia entre variables», construya la distribución conjunta antes de hablar de información mutua.

La frase que describe el problema orienta qué cantidad usar: «un resultado» sugiere autoinformación, «promedio de una fuente» sugiere entropía, «datos bajo un modelo» distingue entropía cruzada de KL, y «dependencia entre variables» exige construir la distribución conjunta antes de recurrir a información mutua.

11 La escala total y el supuesto iid

La entropía expresada en bits por símbolo es una tasa media bajo el modelo. Para una secuencia de n variables independientes e idénticamente distribuidas, la entropía conjunta es la suma de las entropías individuales.

$$H(X_1, \dots, X_n) = \sum_{i=1}^n H(X_i) = nH(X_1) \quad (\text{modelo iid}). \quad (14)$$

Bajo un modelo iid, una fuente de 1,5 bits por símbolo tiene entropía conjunta de 18 bits para 12 símbolos. La etiqueta iid es parte del resultado. En una secuencia con dependencia, la entropía conjunta puede ser menor que $nH(X_1)$ porque observar el contexto reduce incertidumbre. En ACACACAC, por ejemplo, conocer la regla de alternancia hace predecibles casi todas las posiciones aunque la composición marginal sea uniforme.

12 Implementar sin ocultar el razonamiento

ESENCIAL Bash es aquí un medio para convertir una derivación en una traza auditable. Un buen script debe permitir localizar dónde se originó cada número. La salida más útil durante el aprendizaje no es solo $H=1.5$, sino una tabla intermedia con símbolo, conteo, probabilidad y contribución.

```
#!/usr/bin/env bash
set -euo pipefail

if (( $# != 1 )); then
    printf 'usage: %s SEQUENCE\n' "$0" >&2
    exit 64
fi
sequence=$1
if [[ -z "$sequence" ]]; then
    printf 'ERROR: empty sequence\n' >&2
    exit 65
fi
if [[ ! "$sequence" =~ ^[ACGT]+$ ]]; then
    printf 'ERROR: expected uppercase A C G or T: %s\n' "$sequence" >&2
    exit 66
fi

LC_ALL=C awk -v sequence="$sequence" '
function log2(x) { return log(x)/log(2) }
BEGIN {
    n=length(sequence)
    for (i=1; i<=n; i++) count[substr(sequence,i,1)]++
    print "symbol\tcount\tprobability\tcontribution_bits"
    for (i=1; i<=4; i++) {
        symbol=substr("ACGT",i,1)
        if (count[symbol]>0) {
            p=count[symbol]/n
            contribution=-p*log2(p)
            h+=contribution
            printf "%s\t%d\t%.6f\t%.6f\n",symbol,count[symbol],p,contribution
        }
    }
    printf "TOTAL\t%d\t1.000000\t%.6f\n",n,h
}'
```

Para AACG, `entropy_trace.sh` imprime, en orden A,C,G, contribuciones de 0,5 bits y una fila total de 1,5 bits.

12.1 Por qué $\log(x)/\log(2)$

La función `log()` de `awk` calcula el logaritmo natural. La identidad de cambio de base permite obtener base dos mediante $\log(x)/\log(2)$. La razón de dos logaritmos en la misma base es exacta matemáticamente; el valor numérico está sujeto al redondeo de punto flotante.

No es adecuado comparar dos resultados decimales con igualdad textual si se han calculado por rutas numéricas distintas: se declara una tolerancia, por ejemplo 10^{-6} para los ejemplos de este capítulo.

Dos resultados decimales obtenidos por rutas numéricas distintas no se comparan por igualdad textual; se declara una tolerancia explícita, 10^{-6} para los ejemplos de este capítulo.

12.2 Controles mínimos

Antes de confiar en datos reales, conviene usar casos cuyo resultado se conoce:

Constante: AAAA debe producir 0.

Uniforme: ACGT debe producir 2.

Misma composición: AAAACCCC y ACACACAC deben coincidir.

Vacía: debe fallar con un mensaje y código de salida no cero.

Inválida: ACNT debe seguir la política declarada, no desaparecer silenciosamente.

Normalización: la suma de probabilidades debe ser uno dentro de la tolerancia.

Un oráculo prueba una propiedad concreta, no «demuestra que el programa es correcto» en todos los casos. Los tres primeros casos detectan errores comunes de fórmula; los dos siguientes examinan contratos de entrada; la normalización detecta denominadores incoherentes. Hacen falta todos porque fallos distintos pueden producir el mismo número en un ejemplo afortunado.

12.3 Errores de implementación que cambian la ciencia

- **Contar la cabecera FASTA.** Las letras del identificador contaminan el alfabeto y el denominador.
- **Promediar entropías sin ponderar longitudes.** Dos secuencias de 10 y 1000 bases no representan el mismo número de observaciones.
- **Omitir categorías cero en KL cuando P es positivo.** Convierte una divergencia infinita en un número finito incorrecto.
- **Ordenar una tabla por separado.** Si las filas de P y Q dejan de corresponder al mismo símbolo, se comparan categorías distintas.
- **Redondear probabilidades antes de sumar.** La suma puede dejar de ser uno y el error se propaga al logaritmo.
- **Confundir bits por símbolo con bits totales.** El primero es un promedio; el segundo requiere longitud y un modelo conjunto.

Contar letras de una cabecera FASTA junto con la secuencia cambia los conteos y el denominador; en el ejemplo >idA seguido de AAAA, un conteo ingenuo ve siete letras en vez de cuatro símbolos de secuencia.

Promediar sin ponderación entropías de secuencias de longitudes 10 y 1000 con valores 0 y 2 produce 1 bit por símbolo, mientras ponderarlas por número de símbolos produce 1,980198; ambas operaciones responden a preguntas distintas.

Para $P = (0,9, 0,1)$ y $Q = (0,8, 0,2)$, KL es 0,052933 bits si las filas corresponden y 1,652933 bits si las filas de Q se intercambian; ordenar cada tabla por separado puede cambiar la conclusión.

Redondear tres probabilidades $1/3$ a 0,33 antes de sumar produce 0,99: el redondeo intermedio rompe la normalización y debe posponerse hasta presentar el resultado.

13 Límites de inferencia biológica

Una entropía de orden cero no es una anotación funcional: las anotaciones de Gene Ontology relacionan un producto génico y un término con una referencia y un código de evidencia. La medida puede priorizar regiones; no aporta por sí misma esos campos ni la evidencia experimental, filogenética o computacional que los códigos distinguen [6]. El límite es verificable: compare la salida del script, formada por cadena y número, con los componentes exigidos por una anotación.

Una entropía de orden cero escalar no determina una filogenia: cualquier permutación de una secuencia conserva el histograma y la entropía, pero el escalar no conserva patrones de caracteres por taxón ni una topología. Este límite se deriva de TC07-A004; no afirma que la teoría de la información sea inútil en filogenia, sino que esta representación concreta ha descartado la información necesaria para reconstruir una historia.

Evaluar exactitud diagnóstica clínica exige declarar una prueba índice, una condición diana, un estándar de referencia y una población objetivo; un valor aislado de entropía no constituye esa evaluación. STARD define esos elementos y pide medidas como sensibilidad y especificidad [1]. En este capítulo no se propone un uso clínico ni se estima ninguna de ellas.

Los otros dos límites ya tienen mecanismos propios: TC07-A037 demuestra que la simetría de MI no proporciona dirección causal, y TC07-A038 documenta por qué la MI cruda entre columnas no basta para inferir un contacto directo. Descomponerlos evita que una sola cita aparente justificar cuatro inferencias distintas.

13.1 Cuatro preguntas antes de interpretar

1. **¿Qué distribución se midió?** ¿Símbolos individuales, pares, ventanas, columnas de un alineamiento o categorías preprocesadas?
2. **¿Cuál fue la referencia?** En KL, cambiar Q cambia la pregunta y el valor.
3. **¿Qué incertidumbre tiene la estimación?** Tamaño de muestra, dependencia, sesgo y decisiones sobre ceros afectan al resultado.
4. **¿Qué alternativa explicativa existe?** Composición, calidad, ascendencia compartida, cobertura o preprocesamiento pueden producir una señal.

Una conclusión prudente conserva la escala de la evidencia. «Esta ventana tiene menor entropía de orden cero que el fondo bajo el mismo alfabeto y ancho» es una afirmación reproducible. «Esta ventana es un gen» no se sigue de ella. «Estas dos columnas presentan información mutua estimada positiva» describe un cálculo. «Estos residuos contactan directamente» requiere evidencia estructural y análisis de dependencias indirectas.

Una conclusión prudente conserva la escala de la evidencia: una diferencia de entropía de orden cero entre una ventana y el fondo es una afirmación reproducible, pero no implica función biológica, y una información mutua estimada positiva describe un cálculo, no un contacto directo, que requeriría evidencia estructural adicional.

14 Mapa de decisión para resolver problemas

Ante un nuevo ejercicio, puede recorrerse la siguiente secuencia verbal:

1. *Defina la unidad observacional.* Una base, una ventana, una columna o un par de variables no son intercambiables.
2. *Declare el alfabeto y el soporte.* Enumere resultados posibles, incluso los no observados cuando el modelo los permita.
3. *Separe datos y modelo.* Use letras distintas para frecuencias empíricas y probabilidades de referencia.
4. *Formule la pregunta.* Sorpresa individual, incertidumbre media, desajuste o dependencia conducen a medidas distintas.
5. *Anticipe un caso límite.* Constante, uniforme, independencia o cero de soporte proporcionan un oráculo.
6. *Calcule con traza.* Conserve conteos, probabilidades, contribuciones y unidades antes de resumir.
7. *Perturbe una decisión.* Cambie ventana, pseudoconteo o tratamiento de ambiguos y observe la sensibilidad.
8. *Redacte el límite.* Termine diciendo qué información descartó el modelo y qué conclusión no está justificada.

El orden evita que una herramienta decida la pregunta a posteriori. También hace visible la diferencia entre un error de código y una limitación del modelo: si AAAACCCC y ACACACAC coinciden bajo orden cero, el programa puede estar funcionando exactamente como fue definido; lo que falta es contexto en la representación.

15 Síntesis y transición a modelos probabilísticos

La autoinformación convierte probabilidad en sorpresa; la entropía promedia esa sorpresa; la entropía cruzada evalúa datos de una fuente con probabilidades de otro modelo; KL aísla el exceso debido al desajuste; la información mutua compara una distribución conjunta con la independencia. Las cinco cantidades comparten logaritmos y probabilidades, pero responden a preguntas diferentes.

Los ejemplos han mostrado tanto potencia como pérdida. AAAA y ACGT separan concentración y uniformidad. AAAACCCC y ACACACAC demuestran que la composición no conserva el orden. Las ventanas localizan cambios, pero su ancho y solapamiento condicionan el perfil. KL exige dirección y atención al soporte. La información mutua exige verdaderas observaciones emparejadas y no demuestra causalidad.

El siguiente capítulo comienza justo donde termina la entropía de orden cero. Si el símbolo siguiente depende del anterior, una sola distribución marginal es insuficiente. Los modelos de Markov añadirán probabilidades de transición; los HMM añadirán estados no observados que generan símbolos observables. La pregunta ya no será solo «¿qué frecuencias tiene esta secuencia?», sino «¿qué mecanismo probabilístico, con qué dependencias y estados, podría haberla generado?».

Fuentes comentadas y rutas de ampliación

La formulación fundacional de entropía y tasa de información procede de las dos partes publicadas por Shannon [9, 10]. La divergencia dirigida entre distribuciones se apoya en el trabajo de Kullback y Leibler [7]. Para profundizar en propiedades, demostraciones y relaciones entre cantidades, una referencia sistemática es Cover y Thomas [2]. El sesgo y las dificultades de estimar entropía e información mutua a partir de muestras finitas se desarrollan en Paninski [8]. La implementación usa las funciones y reglas documentadas en el manual oficial de GNU awk [4]. Estas fuentes justifican conceptos y métodos; las figuras, ejemplos sintéticos y scripts del capítulo son originales y reproducibles.

Anexo práctico · Auditar información en secuencias

Pregunta, producto y separación de materiales

Dos secuencias pueden compartir composición y entropía de orden cero y, sin embargo, ordenar sus símbolos de manera distinta. Este laboratorio construye una cadena de evidencia para responder: *¿cómo detectar una diferencia local sin presentar la entropía como una anotación biológica?*

El producto individual contiene: el script `analysis_tc07.sh`; su directorio de entrega con resumen, dos perfiles, clasificación y controles; una predicción fechada anterior a la ejecución; una interpretación de 250–400 palabras; y una mini-defensa. El script recibe rutas como argumentos, usa Bash y GNU Awk, funciona sin red y rechaza destinos existentes.

El producto práctico exige script Bash, tablas TSV, controles, predicción previa, interpretación individual y defensa trazable.

El PDF del estudiante no contiene la tabla resuelta, la clasificación del par ni los valores de los perfiles. La clave docente permanece fuera de la distribución y solo se aplica después de la entrega. El analizador de `lib/` es una dependencia fijada, no la solución del script integrador que se evalúa.

El enunciado, el paquete estudiantil y la clave docente son objetos físicamente separados; la raíz distribuida no contiene ni enlaza los oráculos exactos.

Teoría que se recupera.

Este anexo se apoya en cuatro sitios del capítulo y en ninguno más: la fórmula de la entropía, la sección [De una secuencia a una distribución empírica](#) para pasar de conteos a frecuencias, la sección [Entropía por ventanas: localizar sin diagnosticar](#) para elegir la escala de la ventana, y la sección [Límites de inferencia biológica](#) para frenar la inferencia. No se remite a la divergencia ni a la información mutua porque ninguna decisión de esta práctica las utiliza.

Mapa de ruta y productos entregables

La ruta **esencial** conduce al producto completo evaluable. Estudio y referencia son ampliaciones posteriores y no se necesitan para superar el laboratorio. La carga se calcula desde un TSV auditable, no sumando tiempos a ojo.

secciones	ruta	producto	criterio principal
entorno + recuperación Bash	esencial	diagnósticos	trazabilidad, controles
predicción + ejecución guiada	esencial	predicción, resumen	interpretación, cálculo
función + perturbación	esencial	script, perfiles	cálculo
controles + auditoría	esencial	registro, nota adversarial	controles, interpretación
entrega + defensa	esencial	paquete, defensa	trazabilidad, defensa
caso nuevo	estudio	análisis transferido	cálculo
auditoría de implementación	referencia	informe técnico	controles

El mapa ejecutable reparte las secciones del anexo en ruta esencial, de estudio y de referencia; la ruta esencial cubre los cinco criterios de rúbrica.

1 • Entrar en la distribución y verificarla

Desde el directorio del capítulo, monte la raíz estudiantil y compruébela. La raíz es autónoma: contiene el andamiaje de análisis, la biblioteca de apoyo, los datos y el manifiesto que fija sus sumas. No necesita nada de fuera.

```
./practice_assets/create_workspace.sh TC07-student
cd TC07-student
bash check_environment.sh
sha256sum --check --strict manifest.sha256
```

El andamiaje `analysis_tc07.sh` llega con la parte que usted debe escribir marcada como pendiente. El verificador rechaza esa marca: mientras siga ahí, la entrega no se acepta.

La fábrica compila el anexo vigente y publica atómicamente una raíz estudiantil autónoma; el recorrido literal solo consume esa distribución y verifica todos sus hashes.

La prueba de distribución traslada únicamente la raíz entregable a un segundo temporal, impide enlaces de escape y busca nombres, hashes, fragmentos y valores de la clave antes de ejecutar el recorrido.

El diagnóstico comprueba Bash, GNU Awk, `cmp` y las herramientas de archivos requeridas, incluida una comparación funcional, sin depender de la red.

Todos los cálculos C5 invocan explícitamente `gawk`; un ejecutable distinto llamado `awk` no altera el análisis.

El manifiesto de la distribución protege el PDF vigente y los componentes inmutables, pero excluye la plantilla que debe editarse. La fábrica conserva además un manifiesto compuesto privado que fija datos, scripts, clave, parámetros y pruebas de cada artefacto C5.

El manifiesto estudiantil verifica datos, biblioteca, entorno, recuperación y comprobador público antes de calcular.

El manifiesto compuesto C5 enlaza cada reproducción con hashes de datos, scripts, clave docente, prueba y parámetros efectivos.

El FASTA principal distribuido tiene SHA-256:

```
35fac74d29076d48410f2db5ffcd66721d2bc60287ca4c777b9e89c017319953.
```

Los datos son secuencias sintéticas originales, no personales ni clínicas, distribuidas bajo CC0 1.0.

2 • Recuperar Bash antes de leer la plantilla

Este anexo puede distribuirse solo, por lo que no presupone memoria perfecta de capítulos anteriores. Antes de editar, responda: ¿qué diferencia hay entre `$1` y `$#`? ¿cómo se consulta el estado de un comando? ¿por qué se entrecomillan rutas? ¿qué hace una función? ¿por qué un temporal evita dejar una entrega a medias? Si alguna respuesta no es segura, consulte `help functions`, `help test`, `help trap` y `man mktemp`.

Ejecute el diagnóstico de destrezas. No modifica el espacio: usa un directorio temporal y lo elimina al terminar.

```
bash lib/bash_recovery.sh
```

La recuperación ejecutable comprueba funciones, argumentos, rutas, hashes y estados de salida antes de que la plantilla los utilice.

El diagnóstico de destrezas no modifica el espacio de entrega: trabaja en un directorio temporal propio y lo elimina al terminar.

Lea ahora el contrato de `analysis_tc07.sh`: el modo `classify` recibe dos valores y una tolerancia; el modo `analyze` recibe un FASTA y un directorio nuevo. La biblioteca se localiza respecto del propio script, por lo que el espacio puede moverse completo a otra ruta.

`analysis_tc07.sh` expone dos modos: `classify` recibe dos valores y una tolerancia, y `analyze` recibe un FASTA y un directorio nuevo; la biblioteca se localiza respecto del propio script, por lo que el espacio de trabajo puede moverse completo a otra ruta sin romperse.

3 • Conocer los datos y comprometer una predicción

El archivo principal tiene cinco registros sintéticos. Las líneas están envueltas y las cabeceras contienen letras del alfabeto para hacer visible un lector FASTA incorrecto. Antes de ejecutar nada, complete y guarde `prediccion.md` con fecha: longitud, composición esperada, posición relativa de H_0 entre sus cotas, y justificación. No se pide adivinar seis decimales.

registro	longitud	composición	posición de H_0	justificación
constant				
uniform				
blocks				
interleaved				
mixed				

Tarea. Sin IA, derive en `prediccion.md` la entropía de cuatro símbolos equiprobables usando EQ-TC07-ENTROPY. Firme esa predicción antes de ejecutar el resumen.

El cálculo ancla se predice sin IA y se reproduce con Bash antes de interpretar el registro equiprobable.

La evidencia ancla exige una derivación previa y un resultado posterior dentro de las cotas de entropía; la clave exacta no aparece aquí.

Tarea. Prediga qué extremo de la cota alcanza el registro constante y explique por qué aumentar su longitud sin introducir otro símbolo no cambia ese extremo.

El control constante debe alcanzar el extremo de mínima incertidumbre; el estudiante obtiene y registra el valor, no lo copia del PDF.

4 • Ejecución guiada y controles de representación

Solo después de guardar la predicción ejecute:

```
bash lib/tc07_analyze.sh summaries data/tc07_sequences.fasta > resultados/resumen.tsv
cat resultados/resumen.tsv
```

Reconstruya una fila: compruebe que los cuatro conteos suman la longitud, divida por el denominador correcto, verifique que las probabilidades suman uno dentro de 10^{-6} y aplique EQ-TC07-ENTROPY. Compare después con su predicción y anote si el desacuerdo estaba en datos, transformación o interpretación.

El resumen genera una fila por registro del FASTA efectivo; un lector independiente vuelve a derivar longitud y conteos A,C,G,T antes de comprobar normalización y entropía.

El resultado del registro mixto debe caer entre cero y dos bits por símbolo; se contrasta con los conteos derivados del FASTA efectivo, no con una fila prefabricada.

Dos lectores independientes excluyen cabeceras, concatenan líneas y preservan el orden; un FASTA válido generado durante la prueba impide resolver siempre el corpus canónico.

5 • Completar la decisión numérica

Abra el guion de análisis. Hay una única zona marcada como pendiente. La función debe imprimir solo una de las dos etiquetas previstas, ser simétrica al intercambiar los dos valores y rechazar tanto las entradas que no son números como las tolerancias negativas.

La semántica operativa es: comparar la diferencia absoluta con la tolerancia más una guarda de punto flotante de 10^{-12} multiplicada por la mayor escala entre uno, ambos valores y la tolerancia. La guarda absorbe el error binario de los decimales publicados sin aceptar un caso genuinamente exterior.

La clasificación es simétrica y coherente en igualdad, borde por ambos lados, interior y exterior de 10^{-6} ; las entradas no numéricas se rechazan.

No se publica qué etiqueta recibe el par del FASTA. Registre primero su predicción y compruébela después con el modo `classify`. El comprobador público usa relaciones metamórficas y no contiene la tabla docente resuelta.

Tarea. Prediga si `blocks` e `interleaved` serán distinguibles por H_0 . Justifique su predicción desde la composición, no desde el orden visual.

Dos registros con la misma tabla de conteos deben producir la misma entropía de orden cero, aunque su organización posicional difiera.

6 • Perturbar la escala sin sobrescribir

Para longitud n y ancho w , prediga el número de filas mediante $n - w + 1$. Calcule en papel la composición de la primera ventana para dos anchos. No use redirección `>` para guardar tablas: el modo `analyze` escribe todo en un directorio temporal nuevo y solo lo publica si termina completo.

Un perfil exhaustivo de una secuencia de longitud n con ventanas de ancho w contiene $n - w + 1$ filas cuando $1 \leq w \leq n$.

Cambiar el ancho reagrupa símbolos y puede cambiar la entropía de la primera ventana; los valores concretos quedan reservados a la ejecución.

Tarea. Compare anchos 4 y 8. Discuta localización, estabilidad y solapamiento sin afirmar que uno es universalmente mejor. Repita después con `interleaved` y explique qué información recupera el perfil.

El ancho modifica simultáneamente número de ventanas, escala espacial, solapamiento y perfil; la interpretación debe declarar esa elección.

Cuando haya completado la función, genere la entrega mediante el comando literal:

```
bash analysis_tc07.sh analyze data/tc07_sequences.fasta entrega_tc07
```

Esta orden solo funciona cuando la función esté escrita; con el andamiaje tal como llega, termina con error y no crea nada.

Si `entrega_tc07` ya existe, el comando debe fallar y conservarlo. Para repetir, elija otro nombre después de inspeccionar la ejecución anterior; no borre ni sobrescriba evidencia sin decidirlo explícitamente.

Resumen, perfiles y entrega integrada se escriben de forma atómica en destinos nuevos; todo intento de reutilizar el destino falla sin cambiar su hash.

7 · Probar la entrega, no el oráculo

Ejecute el comprobador público parametrizado sobre su propio script:

```
bash check_submission.sh ./analysis_tc07.sh
```

Este comprobador rechaza TODO-TC07 y PENDING, ejecuta ambos modos y fija las etiquetas con valores sintéticos ajenos al FASTA. Prueba igualdad, interior, bordes, exteriores, inversiones y entradas inválidas. Un oráculo Bash/gawk, separado del candidato y sin filas congeladas, relee la ruta efectiva y deriva resumen y perfiles. En cada ejecución se elige una semilla nueva y se crean dos FASTA con longitudes, composición, envoltura, orden e identificadores auxiliares variables. Se imprimen semilla y hashes para repetir el desafío. La prueba docente elige otra semilla después de recibir el candidato.

La prueba contrasta los cinco productos con un oráculo sobre desafíos positivos elegidos en tiempo de ejecución, además de decidir etiquetas, controles, atomicidad y no sobrescritura.

La regresión conserva los mutantes r3/r4 y rechaza además un candidato que solo admite los tres hashes positivos conocidos antes de r5.

El oráculo deriva resumen y ventanas del FASTA efectivo sin reutilizar el analizador ni tablas congeladas.

Público y docente generan dos entradas válidas por semilla, registran datos de replay y exigen que hash y productos sigan cada desafío.

La plantilla es sintácticamente válida y localiza su biblioteca en la copia limpia, pero no supera la prueba de entrega hasta completar el TODO y eliminar PENDING.

Los controles negativos rechazan un símbolo fuera del alfabeto y un registro FASTA sin secuencia mediante estado distinto de cero.

control	propiedad	fallo buscado
extremos conocidos	dentro de cotas	fórmula o base incorrecta
normalización	suma 1 ± 10^{-6}	denominador incoherente
FASTA inválido/vacío	estado no cero	filtrado silencioso
cabecera y orden	representación estable	contaminación/reordenación
tolerancia	simetría, bordes, exterior	punto flotante o igualdad textual
destino existente	fallo e igual hash	sobrescritura silenciosa
entrada alternativa	cambian hash y productos	ignorar la ruta recibida
entrega parametrizada	cero TODO/PENDING	probar el candidato equivocado

La batería cubre procedencia por entrada alternativa, normalización, cotas, cabecera, orden, inválidos, vacíos, tolerancia simétrica, coherencia entre productos, destino existente y candidato real.

El resumen declara ocho campos TSV estables: identificador, longitud, cuatro conteos, suma de probabilidades y H0 en bits.

La prueba de fábrica solo emite PRACTICE_C5_R5_OK después de probar distribución, candidato, oráculo, desafíos runtime reproducibles, mutantes, entorno, controles, no sobrescritura y tolerancia.

El diagnóstico rechaza un cmp hostil antes del comprobador público.

El archivo controls.log no es un mero diagnóstico. Conserva el hash de entrada y un resumen PASS de igualdad, interior, bordes, exteriores, entradas numéricas, FASTA negativos, esquemas y condición de destino nuevo. Si falla un control, la entrega temporal no se publica.

La entrega registra resultados verificables de la batería interna en controls.log; el comprobador público exige cada control y el hash de la entrada efectiva.

8 • Auditar una conclusión engañosa

Considere: «los dos registros obtienen el mismo escalar; por tanto tienen la misma organización y función». Desmonte dato, representación, resultado y conclusión. Use el perfil para investigar orden local, pero no lo convierta en anotación.

eslabón	evidencia disponible	pregunta adversarial
datos	secuencias y metadatos	¿se excluyeron cabeceras e inválidos?
representación	histograma H_0	¿qué relaciones posicionales descarta?
resultado	escalares calculados	¿cambia la lectura con ventanas?
conclusión	comparación limitada	¿qué evidencia requeriría función?

Una coincidencia de entropía de orden cero no establece igual organización, función, anotación ni mecanismo biológico.

Un perfil por ventanas puede impugnar igualdad de organización local, pero no establece por sí solo función biológica.

Tarea. Redacte tres frases: resultado verificable; límite de representación; y siguiente comprobación proporcionada. No añada una promesa que los datos no sostienen.

9 • Entrega, IA y contingencia

Entregue script, carpeta generada, `prediccion.md`, interpretación, comando, versiones y hash. Ante una incidencia, conserve mensaje, estado y último control verde. Sin gnuplot, las tablas siguen siendo la evidencia evaluada.

La representación gráfica es opcional; el producto evaluado se completa íntegramente con tablas locales.

Predicción, primera interpretación y defensa son sin IA; una ayuda posterior de depuración exige registro, contraste local y explicación.

La ruta sin IA usa este PDF, `help`, manuales y pruebas locales. Con ayuda, toda sugerencia se contrasta con el candidato; ninguna fluidez sustituye controles.

10 • Mini-defensa y rúbrica

Tarea. Defienda por qué los controles extremos son necesarios pero insuficientes, qué cambia al modificar el ancho y qué evidencia pediría antes de hablar de función. Prediga en directo el efecto de una perturbación propuesta.

criterio	peso	evidencia observable
corrección computacional	30%	candidato, resumen, perfiles y tolerancia
controles y robustez	25%	negativos, bordes y no sobrescritura
interpretación	25%	resultado, supuesto, límite y siguiente prueba
trazabilidad	10%	predicción, versiones, hashes y comando
defensa individual	10%	explicación y perturbación en directo

La rúbrica asigna 30% a cálculo, 25% a controles, 25% a interpretación, 10% a trazabilidad y 10% a defensa.

La defensa examina controles, sensibilidad al ancho, límites y predicción ante una perturbación.

11 • Estudio, referencia y estado de los PDF

ESTUDIO Diseñe un sexto registro que conserve composición y cambie dependencia local. Prediga resumen y perfil; añada un control sin alterar los datos congelados.

REFERENCIA Audite una función de `lib/tc07_analyze.sh`: entrada, salida, error y operación anti-sobrescritura. Consulte la implementación tras comprometer sus predicciones; la solución queda fuera del paquete.

La fábrica compila `TC07-practice.pdf` desde la fuente vigente; ningún derivado local *superseded* entra en la distribución.

Cierre.

H_0 resume composición y la ventana añade escala espacial; ninguna demuestra función. Predicción, datos, candidato, controles e interpretación deben sostener la misma afirmación.

Bibliografía

- [1] Patrick M. Bossuyt, Johannes B. Reitsma, David E. Bruns, Constantine A. Gatsonis, Paul P. Glasziou, Les M. Irwig, Jeroen G. Lijmer, David Moher, Drummond Rennie, Henrica C. W. de Vet, Herbert Y. Kressel, Nader Rifai, Robert M. Golub, Douglas G. Altman, Lotty Hooft, Daniël A. Korevaar, and Jérémie F. Cohen. STARD 2015: An updated list of essential items for reporting diagnostic accuracy studies. *BMJ*, 351:h5527, 2015.
- [2] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2 edition, 2006.
- [3] Simon D. Dunn, Lindi M. Wahl, and Gregory B. Gloor. Mutual information without the influence of phylogeny or entropy dramatically improves residue contact prediction. *Bioinformatics*, 24(3):333–340, 2008.
- [4] Free Software Foundation. *GNU Awk User's Guide*. GNU Project. GNU Awk User's Guide, nodos Numeric Functions, Arrays y Locales; versión web actualizada el 22-02-2026; consulta: 2026-08-05.
- [5] M. Gardiner-Garden and M. Frommer. CpG islands in vertebrate genomes. *Journal of Molecular Biology*, 196(2):261–282, 1987.
- [6] Gene Ontology Consortium. GO annotations and guide to evidence codes. <https://geneontology.org/docs/go-annotations/>; <https://geneontology.org/docs/guide-go-evidence-codes/>, 2026. Documentación oficial; consulta: 2026-08-05.
- [7] Solomon Kullback and Richard A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951.
- [8] Liam Paninski. Estimation of entropy and mutual information. *Neural Computation*, 15(6):1191–1253, 2003.
- [9] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [10] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(4):623–656, 1948.